Classification: Basic Concepts, Decision Tree Induction, Bayes Classification Methods,

Rule-Based Classification, Classification by Back-propagation, Support Vector Machines, Lazy

Learners, Other Classification Methods.

Classification basic concepts:-

Classification is a task in data mining that involves assigning a class label to each instance in a dataset based on its features. The goal of classification is to build a model that accurately predicts the class labels of new instances based on their features.

There are two main types of classification:

binary classification and multi-class classification.

Binary classification involves classifying instances into two classes, such as “spam” or “not spam”, while multi-class classification involves classifying instances into more than two classes.

The process of building a classification model typically involves the following steps:

Data Collection:
The first step in building a classification model is data collection. In this step, the data relevant to the problem at hand is collected. The data should be representative of the problem and should contain all the necessary attributes and labels needed for classification. The data can be collected from various sources, such as surveys, questionnaires, websites, and databases.

Data Preprocessing:
The second step in building a classification model is data preprocessing. The collected data needs to be preprocessed to ensure its quality. This involves handling missing values, dealing with outliers, and transforming the data into a format suitable for analysis. Data preprocessing also involves converting the data into numerical form, as most classification algorithms require numerical input.

Handling Missing Values: Missing values in the dataset can be handled by replacing them with the mean, median, or mode of the corresponding feature or by removing the entire record.

Dealing with Outliers: Outliers in the dataset can be detected using various statistical techniques such as z-score analysis, boxplots, and scatterplots. Outliers can be removed from the dataset or replaced with the mean, median, or mode of the corresponding feature.

Data Transformation: Data transformation involves scaling or normalizing the data to bring it into a common scale. This is done to ensure that all features have the same level of importance in the analysis.

Feature Selection:
The third step in building a classification model is feature selection. Feature selection involves identifying the most relevant attributes in the dataset for classification. This can be done using various techniques, such as correlation analysis, information gain, and principal component analysis.
Correlation Analysis: Correlation analysis involves identifying the correlation between the features in the dataset. Features that are highly correlated with each other can be removed as they do not provide additional information for classification.

Information Gain: Information gain is a measure of the amount of information that a feature provides for classification. Features with high information gain are selected for classification.

Principal Component Analysis:

Principal Component Analysis (PCA) is a technique used to reduce the dimensionality of the dataset. PCA identifies the most important features in the dataset and removes the redundant ones.

Model Selection:
The fourth step in building a classification model is model selection. Model selection involves selecting the appropriate classification algorithm for the problem at hand. There are several algorithms available, such as decision trees, support vector machines, and neural networks.
Decision Trees: Decision trees are a simple yet powerful classification algorithm. They divide the dataset into smaller subsets based on the values of the features and construct a tree-like model that can be used for classification.

Support Vector Machines: Support Vector Machines (SVMs) are a popular classification algorithm used for both linear and nonlinear classification problems. SVMs are based on the concept of maximum margin, which involves finding the hyperplane that maximizes the distance between the two classes.

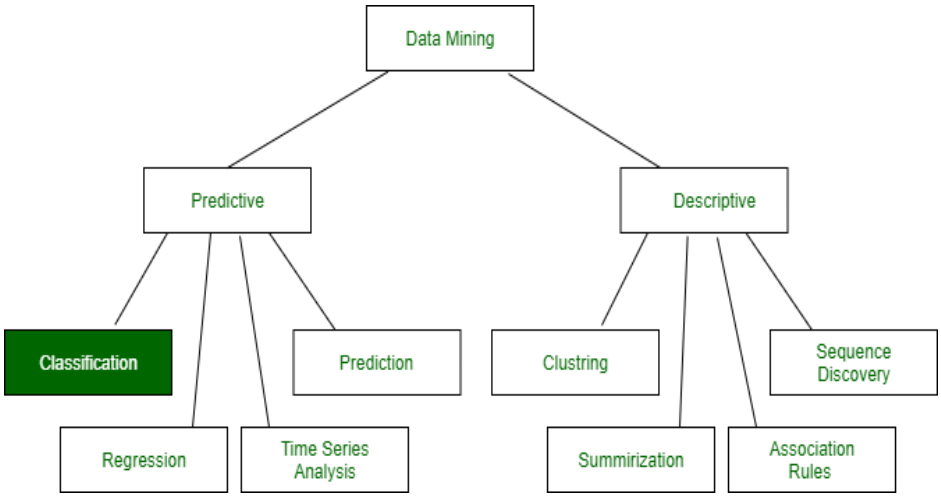
Neural Networks:

Neural Networks are a powerful classification algorithm that can learn complex patterns in the data. They are inspired by the structure of the human brain and consist of multiple layers of interconnected nodes.

Model Training:
The fifth step in building a classification model is model training. Model training involves using the selected classification algorithm to learn the patterns in the data. The data is divided into a training set and a validation set. The model is trained using the training set, and its performance is evaluated on the validation set.

Model Evaluation:
The sixth step in building a classification model is model evaluation. Model evaluation involves assessing the performance of the trained model on a test set. This is done to ensure that the model generalizes well

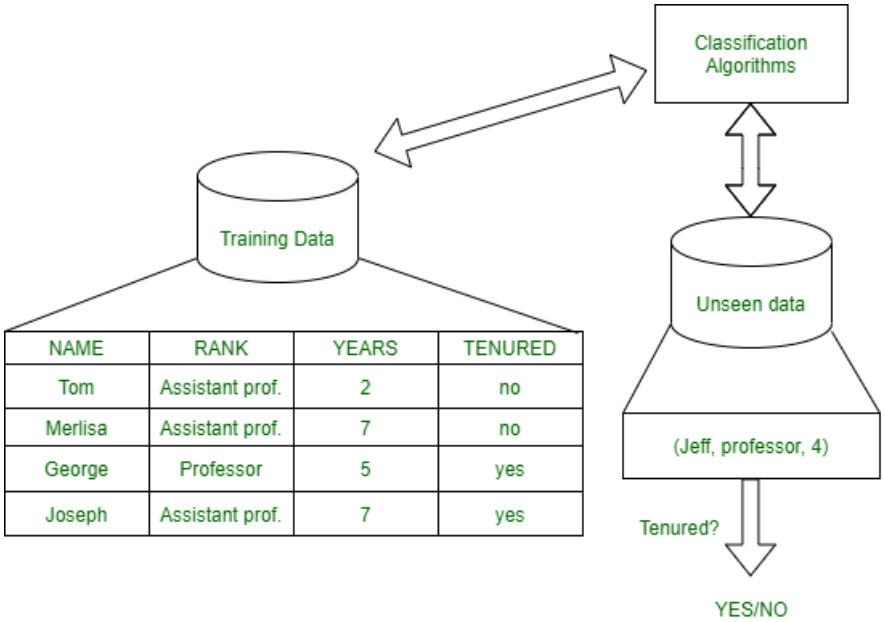
Classification is a widely used technique in data mining and is applied in a variety of domains, such as email filtering, sentiment analysis, and medical diagnosis.



Classification: It is a data analysis task, i.e. the process of finding a model that describes and distinguishes data classes and concepts. Classification is the problem of identifying to which of a set of categories (subpopulations), a new observation belongs to, on the basis of a training set of data containing observations and whose categories membership is known.

Example: Before starting any project, we need to check its feasibility. In this case, a classifier is required to predict class labels such as 'Safe' and 'Risky' for adopting the Project and to further approve it. It is a two-step process such as:

1. **Learning Step (Training Phase)**: Construction of Classification Model
   Different Algorithms are used to build a classifier by making the model learn using the training set available. The model has to be trained for the prediction of accurate results.
2. **Classification Step**: Model used to predict class labels and testing the constructed model on test data and hence estimate the accuracy of the classification rules.



NAME	RANK	YEARS	TENURED
Tom	Assistant prof.	2	no
Merlisa	Assistant prof.	7	no
George	Professor	5	yes
Joseph	Assistant prof.	7	yes

Test data are used to estimate the accuracy of the classification rule

Training and Testing:

Suppose there is a person who is sitting under a fan and the fan starts falling on him, he should get aside in order not to get hurt. So, this is his training part to move away. While Testing if the person sees any heavy object coming towards him or falling on him and moves aside then the system is tested positively and if the person does not move aside then the system is negatively tested.
The same is the case with the data, it should be trained in order to get the accurate and best results.

There are certain data types associated with data mining that actually tells us the format of the file (whether it is in text format or in numerical format).

Attributes – Represents different features of an object. Different types of attributes are:

1. **Binary**: Possesses only two values i.e. True or False
   Example: Suppose there is a survey evaluating some products. We need to check whether it's useful or not. So, the Customer has to answer it in Yes or No.
   Product usefulness: Yes / No
   - **Symmetric**: Both values are equally important in all aspects
   - **Asymmetric**: When both the values may not be important.
2. **Nominal**: When more than two outcomes are possible. It is in Alphabet form rather than being in Integer form.
   Example: One needs to choose some material but of different colors. So, the color might be Yellow, Green, Black, Red.
   Different Colors: Red, Green, Black, Yellow
   - **Ordinal**: Values that must have some meaningful order.
     Example: Suppose there are grade sheets of few students which might contain different

grades as per their performance such as A, B, C, D
Grades: A, B, C, D

- **Continuous**: May have an infinite number of values, it is in float type
  Example: Measuring the weight of few Students in a sequence or orderly manner i.e. 50, 51, 52, 53
  Weight: 50, 51, 52, 53
- **Discrete**: Finite number of values.
  Example: Marks of a Student in a few subjects: 65, 70, 75, 80, 90
  Marks: 65, 70, 75, 80, 90

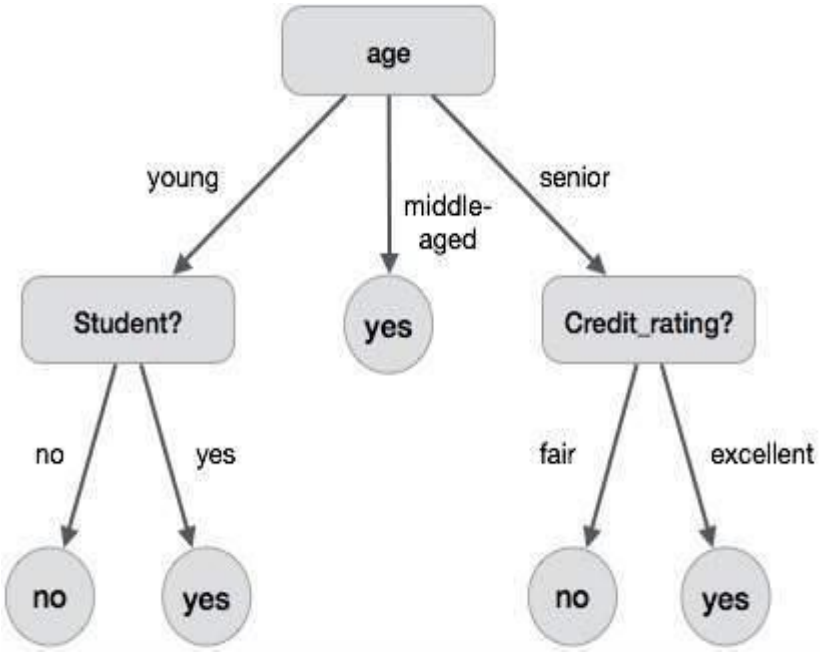
Syntax:

- Mathematical Notation: Classification is based on building a function taking input feature vector "X" and predicting its outcome "Y" (Qualitative response taking values in set C)
- Here Classifier (or model) is used which is a Supervised function, can be designed manually based on the expert's knowledge. It has been constructed to predict class labels (Example: Label – "Yes" or "No" for the approval of some event).

Classifiers can be categorized into two major types:

Discriminative: It is a very basic classifier and determines just one class for each row of data. It tries to model just by depending on the observed data, depends heavily on the quality of data rather than on distributions.
Example: Logistic Regression

1. **Generative**: It models the distribution of individual classes and tries to learn the model that generates the data behind the scenes by estimating assumptions and distributions of the model. Used to predict the unseen data.
   Example: Naive Bayes Classifier
2. A decision tree is a structure that includes a root node, branches, and leaf nodes. Each internal node denotes a test on an attribute, each branch denotes the outcome of a test, and each leaf node holds a class label. The topmost node in the tree is the root node.
3. The following decision tree is for the concept buy_computer that indicates whether a customer at a company is likely to buy a computer or not. Each internal node represents a test on an attribute. Each leaf node represents a class.



The benefits of having a decision tree are as follows −

- It does not require any domain knowledge.
- It is easy to comprehend.
- The learning and classification steps of a decision tree are simple and fast.

Decision Tree Induction Algorithm

A machine researcher named J. Ross Quinlan in 1980 developed a decision tree algorithm known as ID3 (Iterative Dichotomiser). Later, he presented C4.5, which was the successor of ID3. ID3 and C4.5 adopt a greedy approach. In this algorithm, there is no backtracking; the trees are constructed in a top-down recursive divide-and-conquer manner.

```
Generating a decision tree form training tuples of data partition D
Algorithm : Generate_decision_tree

Input:
Data partition, D, which is a set of training tuples
and their associated class labels.
attribute_list, the set of candidate attributes.
Attribute selection method, a procedure to determine the
splitting criterion that best partitions that the data
tuples into individual classes. This criterion includes a
splitting_attribute and either a splitting point or splitting subset.

Output:
 A Decision Tree

Method
create a node N;

if tuples in D are all of the same class, C then
   return N as leaf node labeled with class C;

if attribute_list is empty then
   return N as leaf node with labeled
   with majority class in D;|| majority voting

apply attribute_selection_method(D, attribute_list)
to find the best splitting_criterion;
label node N with splitting_criterion;

if splitting_attribute is discrete-valued and
   multiway splits allowed then  // no restricted to binary trees

attribute_list = splitting attribute; // remove splitting attribute
for each outcome j of splitting criterion

   // partition the tuples and grow subtrees for each partition
   let Dj be the set of data tuples in D satisfying outcome j; // a partition

   if Dj is empty then
      attach a leaf labeled with the majority
      class in D to node N;
   else
      attach the node returned by Generate
      decision tree(Dj, attribute list) to node N;
   end for
return N;
```

Tree Pruning

Tree pruning is performed in order to remove anomalies in the training data due to noise or outliers. The pruned trees are smaller and less complex.

Tree Pruning Approaches

There are two approaches to prune a tree −

- **Pre-pruning** − The tree is pruned by halting its construction early.
- **Post-pruning** - This approach removes a sub-tree from a fully grown tree.

Bayesian classification uses Bayes theorem to predict the occurrence of any event. Bayesian classifiers are the statistical classifiers with the Bayesian probability understandings. The theory expresses how a level of belief, expressed as a probability.

Bayes theorem came into existence after Thomas Bayes, who first utilized conditional probability to provide an algorithm that uses evidence to calculate limits on an unknown parameter.

Bayes's theorem is expressed mathematically by the following equation that is given below.

$$P(X/Y) = \frac{P(Y/X)P(X)}{P(Y)}$$

Where X and Y are the events and P (Y) ≠ 0

P(X/Y) is a **conditional probability** that describes the occurrence of event **X** is given that **Y** is true.

P(Y/X) is a **conditional probability** that describes the occurrence of event **Y** is given that **X** is true.

P(X) and P(Y) are the probabilities of observing X and Y independently of each other. This is known as the **marginal probability**.

Bayesian interpretation:

In the Bayesian interpretation, probability determines a "**degree of belief**." Bayes theorem connects the degree of belief in a hypothesis before and after accounting for evidence. For example, Lets us consider an example of the coin. If we toss a coin, then we get either heads or tails, and the percent of occurrence of either heads and tails is 50%. If the coin is flipped numbers of times, and the outcomes are observed, the degree of belief may rise, fall, or remain the same depending on the outcomes.

For proposition X and evidence Y,

- P(X), the prior, is the primary degree of belief in X
- P(X/Y), the posterior is the degree of belief having accounted for Y.
- The quotient $\frac{P(Y/X)}{P(Y)}$ represents the supports Y provides for X.

Bayes theorem can be derived from the conditional probability:

$$P(X/Y) = \frac{P(X \cap Y)}{P(Y)}, \text{ if } P(Y) \neq 0$$

$$P(Y/X) = \frac{P(Y \cap X)}{P(X)}, \text{ if } P(X) \neq 0$$

Where P (X⋂Y) is the **joint probability** of both X and Y being true, because

$P(Y \cap X) = P(X \cap Y)$

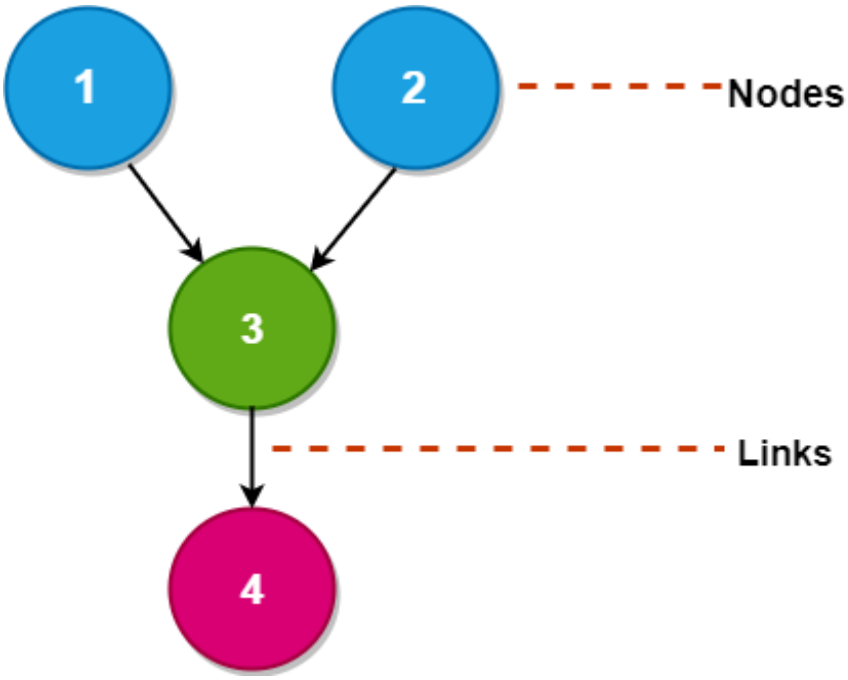
or, $P(X \cap Y) = P(X/Y)P(Y) = P(Y/X)P(X)$

or, $P(X/Y) = \frac{P(Y/X)P(X)}{P(Y)}$, if $P(Y) \neq 0$

Bayesian network:

A Bayesian Network falls under the classification of Probabilistic Graphical Modelling (PGM) procedure that is utilized to compute uncertainties by utilizing the probability concept. Generally known as **Belief Networks, Bayesian Networks** are used to show uncertainties using **Directed Acyclic Graphs** (DAG)

A **Directed Acyclic Graph** is used to show a Bayesian Network, and like some other statistical graph, a DAG consists of a set of nodes and links, where the links signify the connection between the nodes.



The nodes here represent random variables, and the edges define the relationship between these variables.

A DAG models the uncertainty of an event taking place based on the Conditional Probability Distribution (CDP) of each random variable. A **Conditional Probability Table** (CPT) is used to represent the CPD of each variable in a network.

What is Rule-Based Classification?

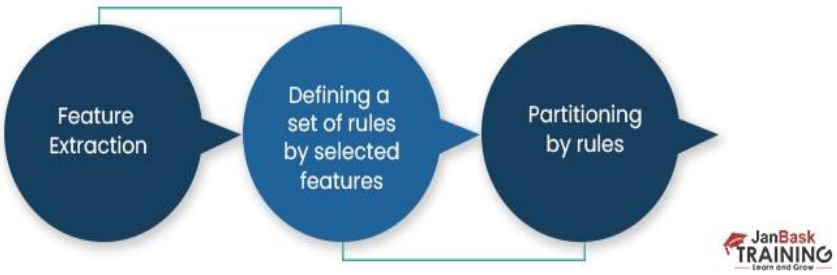
Rule-based classifiers are another method for deducing a subject's categorization; these classifiers establish a subject's classification based on a set of "*if*" and "*otherwise*" rules. The creation of descriptive models normally involves utilising these various classifiers due to the fact that these criteria are easy to comprehend. If a rule is able to anticipate a certain class, we refer to that class as the rule's consequent, and the condition that is evaluated using the if keyword is the rule's antecedent.

Classifiers that rely on rules have the following characteristics:

- The term "*coverage*" is used in the field of statistics to refer to the proportion of available data that satisfies the requirements of a certain rule.
- Rule-based classifiers will often give rules that are not mutually exclusive with one another, which means that several rules may be applicable to the same data set.

- There is a possibility that certain records will not be covered by the rules that are generated by rule-based classifiers.
- They result in linear decision boundaries, but because several rules can be triggered by the same record, they are oftentimes far more involved than a decision tree would be.

Knowing that the rules are not exclusive raises the question of how the class would be picked in the case that many rules, each with possibly different implications, apply to the same data. This raises the question because knowing that the rules are not exclusive raises the question.



The problem described above has two different possible solutions:

1) Both of the rules can be rated, with the rule that has the greatest ranking being the one that decides which class is ultimately appropriate.

2) Even if the criteria aren't organised, we may still distribute votes for each category according to how important they are in comparison to the others.

Using IF-THEN Rules for Classification

Rules are a good way of representing information or bits of knowledge. A rule-based classifier uses a set of IF-THEN rules for classification. An IF-THEN rule is an expression of the form ***IF condition THEN conclusion.***

An example is rule R1,

```
R1: IF age = youth AND student = yes THEN buys computer = yes.
```

The "*IF*"-part (or left-hand side) of a rule is known as the rule antecedent orprecondition. The "THEN"-part (or right-hand side) is the rule consequent. In the rule antecedent, the condition consists of one or more attribute tests (such as age = youth, and student = yes) that are logically ANDed. The rule's consequent contains a class prediction (in this case, we are predicting whether a customer will buy a computer). R1 can also be written as

```
R1: (age = youth) ∧ (student = yes) ⇒ (buys computer = yes).
```

If the condition (that is, all of the attribute tests) in a rule antecedent holds true for a given tuple, we say that the rule antecedent issatisfied (or simply, that the rule is satisfied) and that the rule covers the tuple. A rule R can be assessed by its coverage and accuracy. Given a tuple, X, from a classlabeled data set, D, let ncovers be the number of tuples covered by R; ncorrect be the number of tuples correctly classified by R; and |D| be the number of tuples in D. We can define the coverage and accuracy of R as

```
coverage(R) =ncoversD

accuracy(R) = ncorrectncovers
```

Rule Extraction from a Decision Tree

To classify data, we were taught to construct a decision tree using the data used for training. Decision tree classifiers are widely used because their operation is intuitive and they consistently produce accurate

results. Huge decision trees are often unintelligible. In this part, we examine the process of developing a rule-based classifier by mining a decision tree for IF-THEN rules. For very vast decision trees, the IF-THEN rules may be more intuitive to people.

Every possible branch from the root node to a leaf node must be converted into a rule before the tree can be mined for its rule set. The "IF" element of a rule is formed by logically ANDing all of the route-splitting criteria along that path. The rule's consequent ("*THEN*" section) is the class prediction, which is stored at the leaf node.

Rule Induction Using a Sequential Covering Algorithm

Without initially needing to create a decision tree, IF-THEN rules may be derived from the training data using a sequential covering technique. Each rule for a given class will ideally cover many of the tuples of that class (and presumably none of the tuples of other classes), thus the name "sequential learning." In this part, we'll discuss sequential covering algorithms, the most popular method for mining disjunctive sets of classification rules. Keep in mind that a more recent alternative strategy involves the use of associative classification algorithms to produce classification rules by looking for commonly occurring attribute-value combinations. It's possible that these two things will link together to generate rules that can be utilised for categorization after being studied. Because this later method relies on association rule mining, A wide variety of sequential covering algorithms exist. Variants such as AQ, CN2, and even the more modern RIPPER, have found widespread popularity. This is the overarching plan of attack. Each rule is learnt individually. As soon as a new rule is learnt, the tuples that fall inside its scope are dropped, and the procedure is repeated with the remaining tuples. In contrast to decision tree induction, this sequential learning of rules is more efficient. The induction of a decision tree may be thought of as learning a set of rules all at once, as the path to each leaf in the tree represents a rule.

Each section of the school's regulations is taught separately. If we are trying to learn a rule for class Ci, then ideally that rule would apply to all (or most) of the training tuples in class C and none (or few) of the tuples in other classes. The taught rules should be quite precise in this way. Rule coverage is not need to be comprehensive.

Algorithm:

```
Sequential covering. Learn a set of IF-THEN rules for classification.
Input: D, a data set class-labeled tuples;
Att vals, the set of all attributes and their possible values.
```

Output: A set of IF-THEN rules.

Method: (1) Rule set = {}; // initial set of rules learned is empty

(2) for each class c do

(3) repeat

(4) Rule = Learn One Rule(D, Att vals, c);

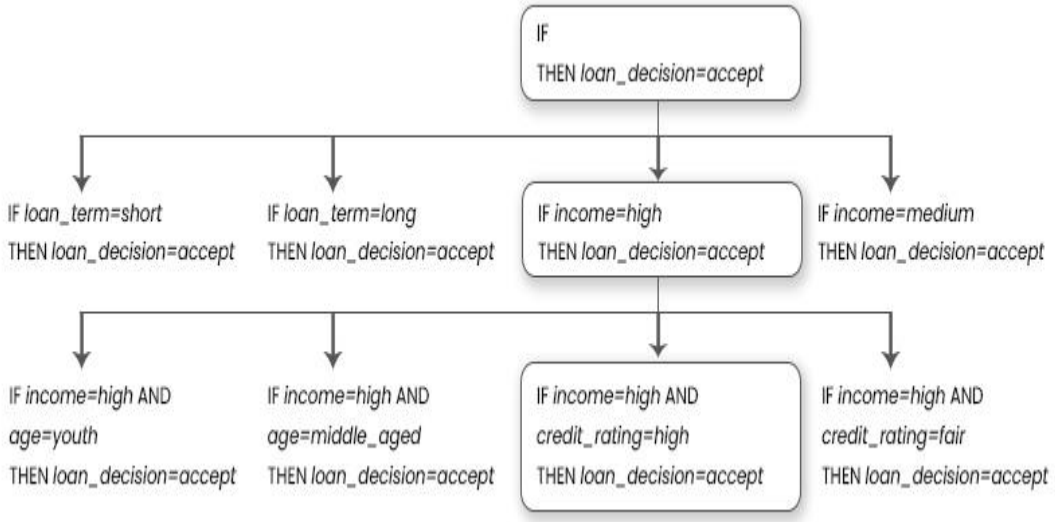
(5) remove tuples covered by Rule from D;

(6) until terminating condition;

(7) Rule set = Rule set +Rule; // add new rule to rule set

(8) endfor

(9) return Rule Set;



Backpropagation in Data Mining

Backpropagation is an algorithm that backpropagates the errors from the output nodes to the input nodes. Therefore, it is simply referred to as the backward propagation of errors. It uses in the vast applications of neural networks in data mining like Character recognition, Signature verification, etc

Backpropagation is a widely used algorithm for training feedforward neural networks. It computes the gradient of the loss function with respect to the network weights. It is very efficient, rather than naively directly computing the gradient concerning each weight. This efficiency makes it possible to use gradient methods to train multi-layer networks and update weights to minimize loss; variants such as gradient descent or stochastic gradient descent are often used.

The backpropagation algorithm works by computing the gradient of the loss function with respect to each weight via the chain rule, computing the gradient layer by layer, and iterating backward from the last layer to avoid redundant computation of intermediate terms in the chain rule.

Features of Backpropagation:

1. it is the gradient descent method as used in the case of simple perceptron network with the differentiable unit.
2. it is different from other networks in respect to the process by which the weights are calculated during the learning period of the network.
3. training is done in the three stages :
    - the feed-forward of input training pattern
    - the calculation and backpropagation of the error
    - updation of the weight

Working of Backpropagation:

Neural networks use supervised learning to generate output vectors from input vectors that the network operates on. It Compares generated output to the desired output and generates an error report if the result does not match the generated output vector. Then it adjusts the weights according to the bug report to get your desired output.

Backpropagation Algorithm:

Step 1: Inputs X, arrive through the preconnected path.

Step 2: The input is modeled using true weights W. Weights are usually chosen randomly.

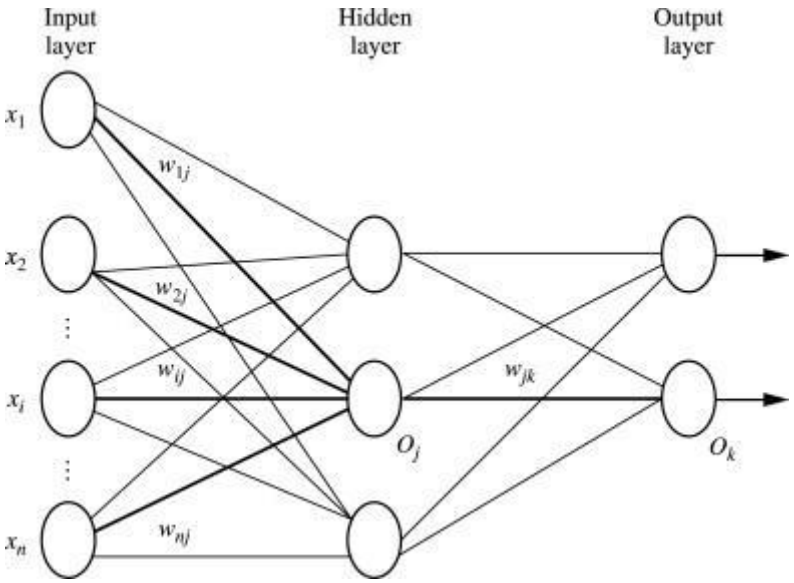
Step 3: Calculate the output of each neuron from the input layer to the hidden layer to the output layer.

Step 4: Calculate the error in the outputs

```
Backpropagation Error= Actual Output - Desired Output
```

Step 5: From the output layer, go back to the hidden layer to adjust the weights to reduce the error.

Step 6: Repeat the process until the desired output is achieved.



Parameters :

- x = inputs training vector $x=(x_1,x_2,\ldots\ldots\ldots\ldots x_n)$.
- t = target vector $t=(t_1,t_2\ldots\ldots\ldots\ldots t_n)$.
- δ_k = error at output unit.
- δ_j = error at hidden layer.
- α = learning rate.
- V_{0j} = bias of hidden unit j.

Training Algorithm :

Step 1: Initialize weight to small random values.

Step 2: While the stepsstopping condition is to be false do step 3 to 10.

Step 3: For each training pair do step 4 to 9 (Feed-Forward).

Step 4: Each input unit receives the signal unit and transmitsthe signal x_i signal to all the units.

Step 5 : Each hidden unit Zj (z=1 to a) sums its weighted input signal to calculate its net input

$$z_{inj} = v_{0j} + \Sigma x_i v_{ij} \quad (\ i=1 \text{ to } n)$$

Applying activation function $z_j = f(z_{inj})$ and sends this signals to all units in the layer about i.e output units

For each output l=unit y_k = (k=1 to m) sums its weighted input signals.

$$y_{ink} = w_{0k} + \Sigma z_i w_{jk} \quad (j=1 \text{ to } a)$$

and applies its activation function to calculate the output signals.

$$y_k = f(y_{ink})$$

Backpropagation Error :

Step 6: Each output unit y_k (k=1 to n) receives a target pattern corresponding to an input pattern then error is calculated as:

$$\delta_k = (t_k - y_k) + y_{ink}$$

Step 7: Each hidden unit Z_j (j=1 to a) sums its input from all units in the layer above

$$\delta_{inj} = \Sigma \delta_j w_{jk}$$

The error information term is calculated as :

$$\delta_j = \delta_{inj} + z_{inj}$$

Updation of weight and bias :

Step 8: Each output unit y_k (k=1 to m) updates its bias and weight (j=1 to a). The weight correction term is given by :

$$\Delta w_{jk} = \alpha \delta_k z_j$$

and the bias correction term is given by $\Delta w_k = \alpha \delta_k$.

therefore $w_{jk(new)} = w_{jk(old)} + \Delta w_{jk}$

$$w_{0k(new)} = w_{ok(old)} + \Delta w_{ok}$$

for each hidden unit z_j (j=1 to a) update its bias and weights (i=0 to n) the weight connection term

$$\Delta v_{ij} = \alpha \delta_j x_i$$

and the bias connection on term

$$\Delta v_{0j} = \alpha \delta_j$$

Therefore $v_{ij(new)} = v_{ij(old)} + \Delta v_{ij}$

$$v_{0j(new)} = v_{0j(old)} + \Delta v_{0j}$$

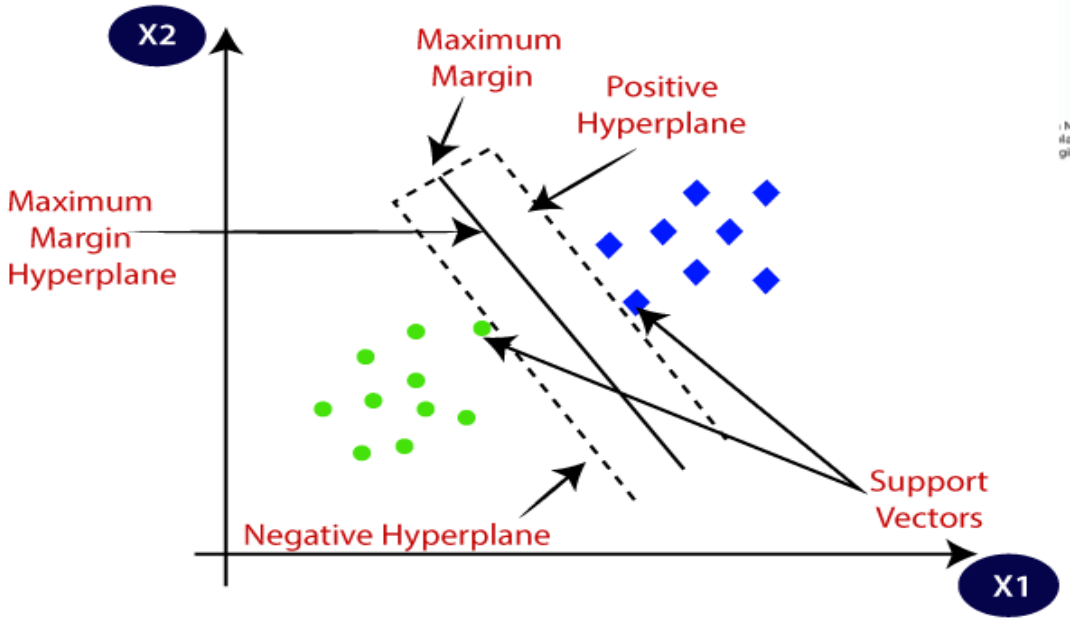
Step 9: Test the stopping condition. The stopping condition can be the minimization of error, number of epochs.

Support Vector Machine Algorithm

Support Vector Machine or SVM is one of the most popular Supervised Learning algorithms, which is used for Classification as well as Regression problems. However, primarily, it is used for Classification problems in Machine Learning.

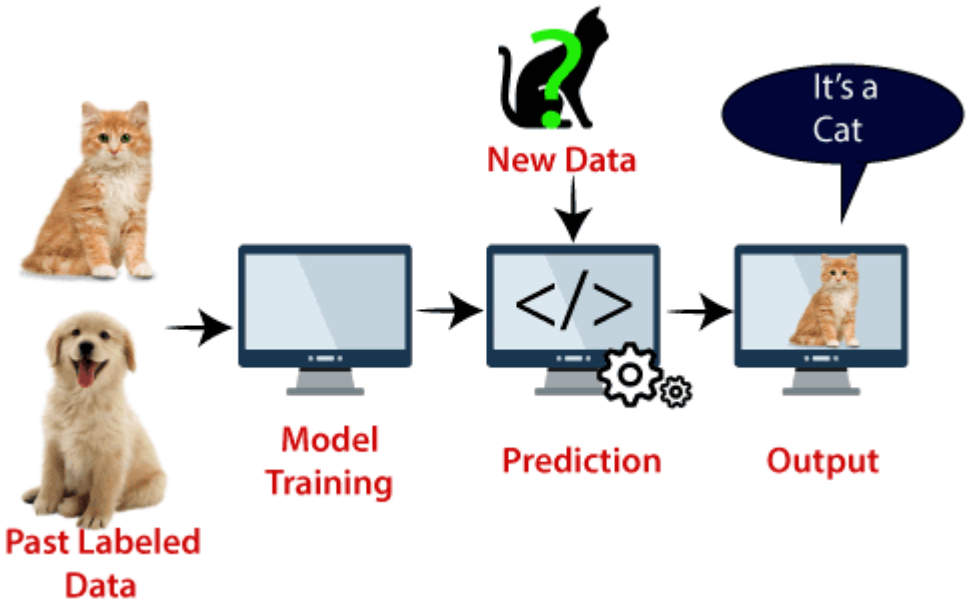
The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane.

SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine. Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane:



Example: SVM can be understood with the example that we have used in the KNN classifier. Suppose we see a strange cat that also has some features of dogs, so if we want a model that can accurately identify whether it is a cat or dog, so such a model can be created by using the SVM algorithm. We will first train our model with lots of images of cats and dogs so that it can learn about different features of cats and dogs, and then we test it with this strange creature. So as support vector creates a decision boundary between these two data (cat and dog) and choose extreme cases (support vectors), it will see the extreme case of cat and dog. On the basis of the support vectors, it will classify it as a cat. Consider the below diagram:

Advertisement



SVM algorithm can be used for **Face detection, image classification, text categorization,** etc.

What is lazy learning?

Lazy learning refers to machine learning processes in which generalization of the training data is delayed until a query is made to the system. This type of learning is also known as Instance-based Learning. Lazy classifiers are very useful when working with large datasets that have a few attributes.

Learning systems have computation occurring at two different times: training time and consultation times.

Training time is the time before the consultation time. During training time, the system derives inferences from training data to prepare for the consultation time.

Consultation time is the time between the moment when an object is presented to a system so that the system can make an inference and the moment when the system finishes making the inference.

In a lazy learning algorithm, most of the computation is done during consultation time. Essentially, a lazy algorithm defers the processing of examples till it receives an explicit request for information.

Why is lazy learning important?

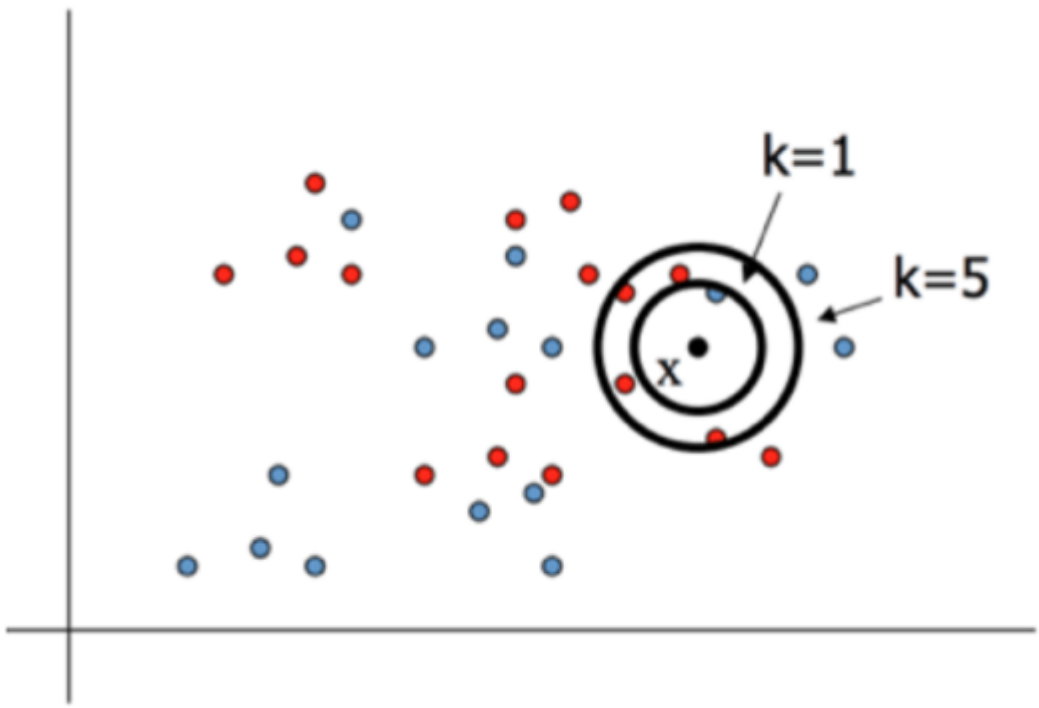
The main purpose behind using lazy learning is that (like in the K-nearest neighbors algorithm which is employed in online recommendation engines like those of Netflix and Amazon) the dataset gets updated with new entries on a continuous basis. Due to these continuous updates, the training data becomes obsolete in a short period of time. So there really is no time to have an actual training phase of some sort.

These lazy algorithms are very beneficial while working with vast, perpetually changing datasets that have a few attributes that are queried quite commonly.

Even if there is an enormous set of attributes available, recommendation queries tend to rely on a relatively smaller set of attributes.

What are some examples of lazy learning?

Instance-based learning, local regression, K-Nearest Neighbors (K-NN), and Lazy Bayesian Rules are some examples of lazy learning.



What is the difference between lazy learning and eager learning?

Lazy learning and eager learning are very different methods. Here are some of the differences:

Lazy learning systems just store training data or conduct minor processing upon it. They wait until test tuples are given to them.

Eager learning systems, on the other hand, take the training data and construct a classification layer before receiving test data.

So while lazy learning systems have a low or non-existent training time and a high consultation time, eager learning systems have a high training time and a low consultation time.

In eager learning, the system needs to commit to a single hypothesis that covers the entire instance space, while in lazy learning, systems can make use of a richer hypothesis space because it employs multiple local linear functions to form its implicit global approximation to the target function.

Essentially, eager learning methods create a general, explicit description of the target function, based on the training examples provided. Lazy learning essentially postpones generalizing beyond the data stored until an explicit request is made.

Note:- Other classification methods notes are provided in separate PDF